

1. Error Audit – Detailed Breakdown

Error	User Stakes (Low/High)	Error Type	Example Scenario	Impact on System & User Experience
Data Ingestion Failure	High	System Limitation	Twitter API rate limits or downtime prevents retrieving tweets.	New tweets are not available for analysis; users receive outdated insights.
Missing/Corrupt Data	High	Background	CSV files may have missing user IDs, tweet content, or timestamps.	Leads to incorrect analysis and incomplete search results.
Duplicate Tweets in Database	Medium	Context	Some tweets may be indexed multiple times due to ingestion errors.	Increases storage use and causes misleading analytics.
Poor Query Results (Irrelevant Tweets Returned)	Medium	Context	Elasticsearch keyword-based search retrieves tweets that are not semantically relevant.	Users may not find useful insights, leading to frustration.
Slow Query Latency (>2 sec response time)	High	System Limitation	Elasticsearch takes too long to retrieve ranked tweets due to high indexing load.	Users experience delays in getting insights, making the tool feel unresponsive.
RAG Model Fails to Generate Insights	High	System Limitation	OpenAI API rate limits or model errors result in no response.	Users see incomplete results or empty responses, affecting trust in the system.
User Confusion Due to Lack of Error Messages	Medium	Context	When an error occurs, no proper message is displayed to guide users.	Users don't understand what went wrong or how to fix it.
Elasticsearch Cluster Overload	High	Background	Large data ingestion or complex queries slow down indexing and retrieval.	Performance degrades over time, affecting all users.

Azure Function Timeouts	High	System Limitation	Long processing tasks (e.g., embedding generation) exceed function execution limits.	System fails to process user queries, causing incomplete responses.
LLM Response Quality is Poor	Medium	Context	The RAG model generates incorrect, nonsensical, or biased responses.	Users receive unreliable insights, reducing trust in the AI system.
Failure in Embedding Generation for Tweets	High	System Limitation	Tweets fail to be converted into vector embeddings due to API failures.	The system cannot perform semantic search, leading to poor query results.
User Interface Freezes During Query Execution	Medium	Context	Frontend UI does not update while waiting for results, causing a poor experience.	Users might think the system has crashed and may abandon the query.
Cache Invalidation Issues	High	Background	Old embeddings or Elasticsearch indices are not updated after reprocessing tweets.	Users may get stale or incorrect results, reducing system reliability.
Incorrect Ranking of Tweets	Medium	Context	The ranking algorithm prioritizes less relevant tweets over highly relevant ones.	Users see low-quality insights, making the tool ineffective.
Frontend Crashes Due to Unexpected Data Inputs	High	System Limitation	Unexpected API responses (e.g., missing fields) cause the UI to break.	Users cannot interact with the system, leading to frustration.
Multiple System Alerts for the Same Error	Medium	Context	Different monitoring tools (Azure, Elasticsearch logs) report the same issue.	Causes confusion for the development team, increasing debugging time.
High Cost Due to Excessive API Calls to OpenAI	High	Background	Frequent calls to the LLM API lead to high operational costs.	Budget overruns could force limits on usage, impacting service availability.

2. Error Source Analysis

Error	Input Error Signals	Relevance Error Signals	System Hierarchy Errors	Failure State
Data Ingestion Failure	Twitter API rate limits prevent ingestion.	No new tweets available, affecting relevance.	Twitter API is an external dependency.	Feature is unusable when no data is ingested.
Missing/Corrupt Data	CSV files contain missing user IDs, timestamps, or tweet content.	Incomplete datasets reduce model accuracy.	Data pipeline may not properly validate schema before ingestion.	Affects indexing and retrieval, leading to inconsistent results.
Duplicate Tweets in Database	System may not properly deduplicate tweets before storage.	Repeated tweets reduce search relevance.	Unclear if duplicate filtering happens at ingestion or query level.	Increases storage use and causes misleading analytics.
Poor Query Results (Irrelevant Tweets Returned)	User queries may not be properly processed for intent matching.	Search engine retrieves keyword matches instead of semantic relevance.	Elasticsearch ranking does not account for user expectations.	Users may not find useful insights, leading to frustration.
Slow Query Latency (>2 sec response time)	Large queries overload the system.	Relevance ranking may be slow due to high data volume.	Elasticsearch indexing and query execution time is not optimized.	System feels unresponsive, reducing usability.
RAG Model Fails to Generate Insights	Model API call fails due to rate limits or timeout.	Incomplete responses degrade insight quality.	OpenAI API is an external system; failures are outside user control.	Users receive empty responses, making the tool ineffective.
User Confusion Due to Lack of Error Messages	No proper feedback provided when a query fails.	Errors are not explained, reducing system transparency.	UI lacks structured error handling.	Users do not know why a failure occurred or how to fix it.
Elasticsearch Cluster Overload	Large-scale indexing operations overwhelm storage.	Query processing slows down due to large index size.	Elasticsearch configuration does not scale well under high load.	Affects all users by slowing retrieval speeds.
Azure Function Timeouts	Long inference times exceed Azure function limits.	API calls stall before results are returned.	Azure serverless infrastructure may not be optimized for workload.	System fails to process user queries, causing incomplete responses.
LLM Response Quality is Poor	Model input prompt may not be optimized.	Model generates vague or incorrect insights.	OpenAI API has inherent limitations in understanding	Users receive unreliable insights, reducing trust in the

			context.	AI system.
Failure in Embedding Generation for Tweets	Tweets may not have clean textual data for embedding.	Embeddings do not accurately capture tweet meaning.	Embedding generation service depends on external AI API.	Semantic search fails, affecting relevance of results.
User Interface Freezes During Query Execution	Query execution is blocking UI updates.	No real-time feedback is given to the user.	Frontend lacks proper async handling for background queries.	Users think the system has crashed and abandon the session.
Cache Invalidation Issues	Old indices are not replaced after reprocessing data.	Stale embeddings cause incorrect search results.	System does not enforce a consistent cache refresh policy.	Users see outdated insights instead of real-time data.
Incorrect Ranking of Tweets	Ranking algorithm does not weigh tweet engagement properly.	Less relevant tweets are prioritized over important ones.	Re-ranking logic may not be dynamically adjusted.	Users see misleading or low-quality insights.
Frontend Crashes Due to Unexpected Data Inputs	API responses contain missing or malformed data.	No validation step before UI renders API response.	UI is tightly coupled with backend output format.	Users cannot interact with the system, leading to frustration.
Multiple System Alerts for the Same Error	Various monitoring tools detect the same failure separately.	Repetitive alerts increase debugging effort.	Logging framework does not aggregate errors properly.	Developers take longer to resolve issues due to excessive noise.
High Cost Due to Excessive API Calls to OpenAI	Frequent LLM requests cause unnecessary expenditure.	Over-reliance on API calls for minor queries.	No caching strategy to optimize repeated requests.	Budget overruns may limit service availability for all users.

3. Error Resolution

1. Data Ingestion Failure

- Error Rationale:
 - Users expect fresh tweets, but the system fails to ingest new data due to API rate limits or downtime.
- Solution Type:
 - User Control: Implement retry logic and backup data sources.
 - Feedback: Provide real-time notifications about ingestion status.

- Error Resolution:
 - User Path: Users receive a notification about ingestion delays and an option to retry after a cooldown period.
 - Opportunity for Model Improvement:
 - Implement a batch ingestion mechanism to optimize API requests.
 - Use incremental updates instead of full re-indexing to reduce API usage.

2. Missing/Corrupt Data

- Error Rationale:
 - Users notice incomplete or missing information in the retrieved tweets.
- Solution Type:
 - Feedback: Display warnings when certain data fields are unavailable.
 - Other: Implement data validation checks before indexing tweets.
- Error Resolution:
 - User Path: Users receive a message explaining missing data and an option to retry or proceed with partial results.
 - Opportunity for Model Improvement:
 - Implement schema validation before ingesting data.
 - Log missing fields for debugging and corrective actions.

3. Duplicate Tweets in Database

- Error Rationale:
 - Users see the same tweet multiple times, making the search results feel redundant.
- Solution Type:
 - Other: Implement de-duplication logic before indexing.
- Error Resolution:
 - User Path: Users can report duplicate tweets, which will be flagged for removal.
 - Opportunity for Model Improvement:
 - Improve deduplication logic by checking tweet IDs and content similarity before indexing.
 - Use hashing techniques to detect and remove duplicate entries.

4. Poor Query Results (Irrelevant Tweets Returned)

- Error Rationale:
 - Users expect relevant tweets but receive unrelated or low-quality results.
- Solution Type:
 - User Control: Allow users to refine searches using filters.
 - Feedback: Provide a feedback mechanism to report irrelevant results.
- Error Resolution:
 - User Path: Users can rate results and provide feedback on relevance.
 - Opportunity for Model Improvement:
 - Train the ranking model using engagement metrics (likes, retweets).
 - Implement semantic search using Vector DB embeddings instead of pure keyword matching.

5. Slow Query Latency (>2 sec response time)

- Error Rationale:
 - Users experience delays, making the system feel slow and unresponsive.
- Solution Type:
 - Other: Optimize Elasticsearch indexing and caching.
- Error Resolution:
 - User Path: Users see a loading indicator and receive suggestions to refine queries for faster results.
 - Opportunity for Model Improvement:
 - Use query caching for frequent searches.
 - Implement asynchronous processing for large-scale queries.

6. RAG Model Fails to Generate Insights

- Error Rationale:
 - Users receive an empty or incomplete response when expecting insights.
- Solution Type:
 - Feedback: Notify users about rate limits or API failures.
 - User Control: Provide alternative query options.
- Error Resolution:
 - User Path: Users see a message explaining the issue and an option to retry with a different model or a cached response.
 - Opportunity for Model Improvement:
 - Implement fallback mechanisms (e.g., switching to another LLM provider).
 - Cache frequent responses to reduce API load.

7. User Confusion Due to Lack of Error Messages

- Error Rationale:
 - Users don't understand why something failed.
- Solution Type:
 - Feedback: Improve error messages and tooltips.
- Error Resolution:
 - User Path: Users receive clear error descriptions and potential solutions.
 - Opportunity for Model Improvement:
 - Implement detailed logging and user-friendly messages.

8. Elasticsearch Cluster Overload

- Error Rationale:
 - Search queries take too long or fail due to system overload.
- Solution Type:
 - Other: Optimize Elasticsearch sharding and indexing strategies.
- Error Resolution:
 - User Path: Users see a message recommending a more specific query to reduce load.

- Opportunity for Model Improvement:
 - Use distributed indexing and query partitioning to scale Elasticsearch efficiently.

9. Azure Function Timeouts

- Error Rationale:
 - Model inference takes too long, causing API failures.
- Solution Type:
 - Other: Optimize function execution time and batch processing.
- Error Resolution:
 - User Path: Users are notified of delays and given an option to queue the request asynchronously.
 - Opportunity for Model Improvement:
 - Use micro-batching instead of processing large requests at once.

10. LLM Response Quality is Poor

- Error Rationale:
 - Users get vague, misleading, or biased responses.
- Solution Type:
 - Feedback: Allow users to rate responses and report issues.
- Error Resolution:
 - User Path: Users can flag responses for review, improving the model's future accuracy.
 - Opportunity for Model Improvement:
 - Fine-tune the LLM based on user feedback.

11. High Cost Due to Excessive API Calls to OpenAI

- Error Rationale:
 - Too many API calls drive up costs.
- Solution Type:
 - Other: Implement query caching and cost-aware API usage.
- Error Resolution:
 - User Path: Users are notified when querying cached responses instead of making an expensive API call.
 - Opportunity for Model Improvement:
 - Implement query batching and request optimization.

12. Incorrect Ranking of Tweets

- Error Rationale:
 - Less relevant tweets are ranked higher.
- Solution Type:
 - User Control: Allow users to sort results by different relevance criteria.
- Error Resolution:
 - User Path: Users can manually reorder or filter results.

- Opportunity for Model Improvement:
 - Improve ranking algorithm by incorporating engagement metrics.

2. Quality Assurance

Objective

To ensure continuous monitoring and resolution of system errors, improving performance and user experience. The system will be tested and criticized early and often using different feedback mechanisms.

Goal	Method	Review Frequency	Start Date	Review / End Date
Ensure Data Integrity	Validate data schema, check for missing/corrupt data	Daily	Week 1	Ongoing
Monitor Model Performance	Evaluate query relevance, precision/recall scores	Weekly	Week 2	Ongoing
Improve Error Messaging	Collect user feedback on unclear or missing error messages	Monthly	Week 3	Ongoing
Optimize Query Latency	Track Elasticsearch response time and system performance	Daily	Week 4	Ongoing
Reduce API Costs	Implement request caching and monitor API call rates	Weekly	Week 5	Ongoing

Enhance User Experience	Collect feedback via in-product surveys	Monthly	Week 6	Ongoing
-------------------------	---	---------	--------	---------

QA Implementation & Monitoring Tools

To ensure effective tracking and resolution of errors, the following tools will be utilized:

- **Azure Monitor & Kibana:** Logs function execution times, failures, and errors.
- **Elasticsearch Query Logs:** Tracks search query performance and relevance.
- **OpenAI API Logs:** Detects failed responses, timeouts, and rate limits.
- **Google Analytics & Hotjar:** Captures user interactions and identifies usability issues.
- **User Feedback Surveys (Google Forms, Typeform):** Collects qualitative user feedback.
- **GitHub Issues & JIRA:** Logs and tracks bug reports from developers and users.

Expected Outcomes

- **Early Detection of Failures:** Faster response to system breakdowns via real-time monitoring.
- **Improved Query Accuracy:** Continuous model tuning based on user feedback.
- **Optimized Performance:** Reduced query latency and better search rankings.
- **User Satisfaction:** Clearer error messages and improved search relevance.
- **Lower Costs:** Optimized API usage and reduced infrastructure expenses.